

Pattern Recognition Project

Burnout Risk Prediction

Project Discussion Report

Course	Pattern Recognition
Dataset	Work Productivity.csv
Target Variable	Burnout_Risk (0 = No, 1 = Yes)
Task Type	Binary Classification

Project Links

Code Repository (Colab): [Work Productivity / Burnout Risk](#)

Dataset Link (Drive): [Work Productivity.csv](#)

Doctor Name: Dr. Yasmine Mahmoud

Teaching Assistant Name: Eng. Heba Mohamed

Team Members:

- Abdelrahman Emad Mahmoud (2300061)
- Amr khalid Ahmed (2300542)
- Gamila Refaat Fawzy (2300506)
- Dalia mohamed adel (2300517)

1. Project Overview

This project applies machine learning classification to predict whether an employee is at risk of burnout based on their work habits, environment, and personal factors.

We follow the full machine learning pipeline: data loading, preprocessing, dimension reduction using multiple techniques, model training and evaluation (before and after reduction), and hyperparameter tuning using GridSearchCV.

Four models are used: Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine (SVC). Additionally, PCA (Principal Component Analysis) is used as a feature extraction approach with Logistic Regression.

1.1 Dataset

The dataset used is Work Productivity.csv. It contains employee records with features related to their job, work habits, lifestyle, and work environment. The target column is Burnout_Risk (Yes/No), which is converted to binary (1/0).

Feature	Description
Employee_ID	Unique identifier — dropped (not useful for prediction)
Productivity_Score	Direct score — dropped (too closely related to target, causes data leakage)
Age, Experience_Years	Numeric employee characteristics
Gender, Country, Job_Role	Categorical demographic features
Company_Size, Work_Environment	Categorical workplace features
Work_Hours_Per_Day, Meetings_Per_Day	Numeric work habit features
Internet_Speed_Mbps	Numeric infrastructure feature
Sleep_Hours, Exercise_Hours_Per_Week	Numeric lifestyle features
Screen_Time_Hours	Numeric daily screen usage
Stress_Level	Ordinal: Low=0, Medium=1, High=2
Burnout_Risk	TARGET: No=0, Yes=1 (binary classification)

2. Cell 1–2: Imports & Data Loading

Cell 2 — Import Libraries & Load Dataset

What this cell does

This is the foundation of the entire project. It imports all required Python libraries for data manipulation, visualization, machine learning, and preprocessing. It also loads the dataset and drops two columns that would either cause data leakage or provide no predictive value.

Libraries imported and why

Library	Purpose
<code>pandas (pd)</code>	Load, manipulate, and analyze the CSV dataset as a DataFrame
<code>numpy (np)</code>	Numerical operations, array handling, quantile calculations
<code>seaborn / matplotlib</code>	Visualize data: boxplots, confusion matrices, bar charts, heatmaps
<code>sklearn.model_selection</code>	<code>train_test_split</code> , <code>GridSearchCV</code> — split data and tune hyperparameters
<code>sklearn.preprocessing</code>	<code>StandardScaler</code> , <code>MinMaxScaler</code> , <code>OneHotEncoder</code> , <code>LabelEncoder</code> — normalize and encode features
<code>sklearn.compose / pipeline</code>	<code>ColumnTransformer</code> and <code>Pipeline</code> — combine preprocessing steps cleanly
<code>sklearn.linear_model</code>	<code>LogisticRegression</code> — first classification model
<code>sklearn.tree</code>	<code>DecisionTreeClassifier</code> — second classification model
<code>sklearn.ensemble</code>	<code>RandomForestClassifier</code> — third classification model
<code>sklearn.svm</code>	<code>SVC</code> — fourth classification model (Support Vector Classifier)
<code>sklearn.decomposition</code>	<code>PCA</code> — feature extraction technique (dimensionality reduction)
<code>sklearn.feature_selection</code>	<code>SelectKBest</code> , <code>chi2</code> , <code>f_classif</code> , <code>VarianceThreshold</code> — feature selection techniques
<code>sklearn.metrics</code>	<code>accuracy_score</code> , <code>confusion_matrix</code> , <code>classification_report</code> — model evaluation
<code>imblearn.over_sampling</code>	<code>SMOTE</code> — handle class imbalance in Random Forest section

Code

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.feature_selection import SelectKBest, chi2, VarianceThreshold

df = pd.read_csv("Work Productivity.csv")
df = df.drop(["Employee_ID", "Productivity_Score"], axis=1)
```

Why we drop Employee_ID and Productivity_Score

Employee_ID: A unique identifier with no relationship to burnout risk. Including it would give the model meaningless noise.

Productivity_Score: This is a direct numerical score that is essentially a summary of the target variable. Keeping it would cause data leakage — the model would use information that directly encodes the answer, making accuracy artificially high and useless in real scenarios.

3. Cell 4: Preprocessing

Cell 4 — Data Cleaning, Encoding, Outlier Removal

Preprocessing is the most critical step in any machine learning project. The quality of your data directly determines the quality of your model. This cell handles six preprocessing tasks in sequence.

Step 1 — Missing Value Check & Removal

We first check how many values are missing in each column using `df.isnull().sum()`. Any rows that contain missing values are removed using `df.dropna()`. This ensures all models receive complete records and avoids errors during training.

```
print("Missing Values:", df.isnull().sum())
df = df.dropna()
```

Step 2 — Duplicate Check & Removal

Duplicate rows can cause a model to overfit by learning from repeated examples. We check how many duplicate rows exist and remove them, ensuring every data point is unique.

```
print("Duplicates:", df.duplicated().sum())
df = df.drop_duplicates()
```

Step 3 — Ordinal Encoding

`Stress_Level` contains meaningful order (Low < Medium < High), so we encode it as 0, 1, 2 using a manual map. Similarly, `Burnout_Risk` (our target) is encoded as 0 (No) and 1 (Yes). We use manual mapping instead of `LabelEncoder` here to preserve the ordinal relationship.

```
df["Stress_Level"] = df["Stress_Level"].map({"Low": 0, "Medium": 1, "High": 2})
df["Burnout_Risk"] = df["Burnout_Risk"].map({"No": 0, "Yes": 1})
df = df.dropna() # drop any rows that failed the mapping
```

Step 4 — Identify Numeric Columns

We identify all numeric columns (excluding the target `Burnout_Risk`) for outlier removal. This avoids accidentally applying IQR filtering to the target or categorical columns.

```
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
numeric_cols = [col for col in numeric_cols if col != "Burnout_Risk"]
```

Step 5 — Outlier Removal using IQR

Outliers can distort model training, especially for distance-based models like SVC and normalization-sensitive models like Logistic Regression. We use the Interquartile Range (IQR) method:

- Calculate Q1 (25th percentile) and Q3 (75th percentile) for each column
- $IQR = Q3 - Q1$
- Any row where a value is below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is removed

In the fixed version, a boxplot is generated BEFORE and AFTER outlier removal so that the effect can be visualized. This is important for the report and discussion — it shows the reviewer you understood what outliers were present and how they were handled.

```
def remove_outliers(df, cols):
    for col in cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower) & (df[col] <= upper)]
    return df

print("Shape before outliers:", df.shape)
df = remove_outliers(df, numeric_cols)
print("Shape after outliers:", df.shape)
```

Step 6 — Define X and y (Feature Matrix and Target)

After all cleaning is done, we split the DataFrame into X (all feature columns) and y (the target column Burnout_Risk). This is the standard supervised learning setup where X is what the model learns from and y is what it predicts.

```
X = df.drop("Burnout_Risk", axis=1)
y = df["Burnout_Risk"]
```

4. Correlation Matrix (Dimension Reduction Insight)

Added Section — Correlation Heatmap

Before applying any feature selection algorithm, we use a correlation matrix to visually understand the relationships between features and the target variable. This helps justify which features are likely to be important and detects multicollinearity (features that are too similar to each other).

How it works

- One-hot encode all categorical columns to make them numeric using `pd.get_dummies()`
- Compute the pairwise Pearson correlation between all columns using `df.corr()`
- Visualize as a heatmap using seaborn — values near +1 mean strong positive correlation, -1 means strong negative, 0 means no relationship
- Extract the column for `Burnout_Risk` and sort by absolute correlation to find the most informative features

```
X_for_corr = pd.get_dummies(X, drop_first=True)
corr_df = pd.concat([X_for_corr, y], axis=1)

plt.figure(figsize=(18, 14))
corr_matrix = corr_df.corr()
sns.heatmap(corr_matrix, annot=False, cmap="coolwarm", center=0)
plt.title("Correlation Matrix - All Features")
plt.show()

# Top correlations with target
target_corr =
corr_matrix["Burnout_Risk"].drop("Burnout_Risk").abs().sort_values(ascending=False)
print("Top 10 features correlated with Burnout_Risk:")
print(target_corr.head(10))
```

Why this matters for the discussion: The correlation matrix is explicitly listed as a required dimension reduction technique. It gives a data-driven justification for feature selection. If a reviewer asks 'why did you pick those features?', you can refer back to this heatmap and the top-10 correlation table.

5. Variance Threshold (Feature Selection)

Added Section — VarianceThreshold Applied

VarianceThreshold is a filter-based feature selection technique that removes features whose variance is below a defined threshold. Features with near-zero variance carry almost no information because they are nearly constant across all samples — including them wastes model capacity and adds noise.

Why we use threshold = 0.01

A threshold of 0.01 means we remove any feature where 99% of values are the same. This is conservative enough to keep all genuinely informative features while removing truly constant or near-constant columns.

```
X_encoded = pd.get_dummies(X, drop_first=True)

print("Features BEFORE Variance Threshold:", X_encoded.shape[1])

vt = VarianceThreshold(threshold=0.01)
X_vt = vt.fit_transform(X_encoded)

print("Features AFTER Variance Threshold:", X_vt.shape[1])
print("Features removed:", X_encoded.shape[1] - X_vt.shape[1])

# Visualize variances
variances = pd.Series(vt.variances_, index=X_encoded.columns).sort_values()
plt.figure(figsize=(14, 5))
variances.plot(kind="bar")
plt.axhline(y=0.01, color="red", linestyle="--", label="Threshold = 0.01")
plt.title("Feature Variances")
plt.legend()
plt.show()
```

6. Logistic Regression — Chi-Square Feature Selection

Cell 5–6 — Logistic Regression (Abdelrahman Emad)

6.1 What is Logistic Regression?

Logistic Regression is a linear classification algorithm. Despite its name, it is a classification model, not a regression model. It calculates the probability that a given input belongs to class 1 (Burnout Risk = Yes) using the sigmoid function, and predicts the class with higher probability.

Formula: $P(Y=1 | X) = 1 / (1 + e^{-(z)})$ where $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$

The model learns the weights (w) during training. The C parameter controls regularization strength — smaller C = stronger regularization = simpler model.

6.2 Chi-Square Feature Selection

Chi-Square (χ^2) is a statistical test that measures the dependency between a feature and the target variable. A high chi-square score means the feature is statistically dependent on the target — it changes in a non-random way when the class changes. We use `SelectKBest` with `chi2` to keep the top 10 features.

Important: Chi-Square requires non-negative values. We apply `MinMaxScaler` first to shift all values to the $[0, 1]$ range before computing chi-square scores, then apply `StandardScaler` afterward for logistic regression training.

6.3 Model BEFORE Dimension Reduction

A standard Logistic Regression is trained on all features (after one-hot encoding and `StandardScaler`). This is our baseline accuracy — what we get without any feature selection.

6.4 Model AFTER Dimension Reduction + GridSearchCV

We apply `SelectKBest(chi2, k=10)` to reduce features, then run `GridSearchCV` to find the best combination of `C` and solver. `GridSearchCV` tries every combination across 5-fold cross-validation and selects the one with the highest mean accuracy.

Hyperparameter Grid

```
param_grid = {  
    "C": [0.01, 0.1, 1, 10],          # Regularization strength  
    "solver": ["liblinear", "lbfgs"] # Optimization algorithm  
}  
grid = GridSearchCV(LogisticRegression(max_iter=1000), param_grid, cv=5,  
                    scoring="accuracy")
```

Output Visualizations

- Bar chart: Accuracy before vs after Chi-Square reduction
- Confusion matrix heatmap: Shows TP, TN, FP, FN for the after model
- Feature coefficient chart: Shows which features push the prediction toward or away from burnout risk
- User prediction: Runs a custom employee profile through the trained model

7. Decision Tree — Chi-Square Feature Selection

Cell 7–8 — Decision Tree (Gamila Refaat)

7.1 What is a Decision Tree?

A Decision Tree is a non-linear classifier that splits the data into branches based on feature conditions. Each internal node asks a question (e.g., Sleep_Hours < 6?), each branch is a possible answer, and each leaf node gives a final class prediction. Trees are easy to interpret but can overfit if not constrained.

7.2 Pipeline with ColumnTransformer

This cell uses sklearn Pipelines to chain preprocessing and modeling into one object. This is a best practice because it prevents data leakage — the scaler fits only on training data, not the test set. The ColumnTransformer applies different preprocessing to different columns:

- Numerical columns: MinMaxScaler (scales to [0,1])
- Categorical columns: OneHotEncoder (converts categories to binary dummy columns, drop='first' to avoid dummy variable trap)

7.3 Chi-Square Selection Inside the Pipeline

SelectKBest(chi2) is added as a pipeline step between preprocessing and the classifier. This is cleaner than applying it manually — the entire pipeline can be tuned with GridSearchCV, including the k parameter.

Hyperparameter Grid

```
param_grid = {
    "feature_selection__k":      [5, 10, 15],      # Number of features to keep
    "classifier__max_depth":     [5, 10, 15],      # Max tree depth (prevents
overfitting)
    "classifier__min_samples_split": [2, 5, 10]    # Min samples to split a node
}
```

Output Visualizations

- Bar chart: Accuracy before vs after Chi-Square
- Confusion matrix: Heatmap of prediction results (Oranges colormap)
- User prediction: Custom employee profile through the pipeline

8. Random Forest — f_classif Feature Selection + SMOTE

Cell 9–10 — Random Forest (Amr Khalid)

8.1 What is Random Forest?

Random Forest is an ensemble learning method that builds multiple Decision Trees during training and merges their predictions. Each tree is trained on a random subset of the data (bootstrap sampling) and uses a random subset of features at each split. The final prediction is the majority vote across all trees. This reduces variance and overfitting compared to a single tree.

8.2 LabelEncoder for Categorical Columns

Random Forest handles mixed data well but requires numeric input. We use LabelEncoder (instead of OneHotEncoder) to convert categorical columns to integers. The encoder for each column is saved in a dictionary so it can be reused to transform user-input data at prediction time.

8.3 SMOTE — Handling Class Imbalance

SMOTE (Synthetic Minority Over-sampling Technique) generates synthetic samples of the minority class (in this case, Burnout_Risk = 1 if it has fewer samples). This prevents the model from being biased toward the majority class.

CRITICAL FIX applied in the fixed version: SMOTE is now applied consistently in BOTH the before and after models. In the original notebook, SMOTE was only applied after reduction, which made the accuracy comparison unfair — the before model had a class imbalance but the after model did not.

SMOTE must only be applied to the TRAINING set — never the test set. Applying it to the test set would introduce artificial samples into evaluation.

8.4 f_classif (ANOVA F-test) Feature Selection

f_classif computes the Analysis of Variance (ANOVA) F-statistic between each feature and the target. A high F-value means the feature's mean differs significantly between the two classes (burnout vs no burnout), making it a strong predictor. Unlike chi-square, f_classif works well with continuous numerical features.

Hyperparameter Grid

```
param_grid = {  
    "n_estimators": [50, 100],      # Number of trees in the forest  
    "max_depth":    [5, 10, None]   # None = grow full trees  
}
```

Output Visualizations

- Bar chart: Accuracy before vs after f_classif reduction
- Confusion matrix: Heatmap (Purples colormap)
- Feature importance chart: Horizontal bar chart showing each selected feature's contribution to the model
- Classification report: Precision, recall, F1-score per class

9. Support Vector Classifier (SVC) — f_classif Feature Selection

Cell 11–12 — SVC (Dalia Mohamed)

9.1 What is SVC?

Support Vector Classifier finds the optimal hyperplane that maximally separates the two classes. The 'support vectors' are the data points closest to the decision boundary. The RBF (Radial Basis Function) kernel maps the data into a higher-dimensional space to find non-linear separations.

C parameter: Controls the trade-off between a smooth decision boundary and classifying training points correctly. High C = low bias, high variance. Low C = high bias, low variance.
gamma: Controls how far the influence of a single training example reaches. High gamma = close influence = complex boundary. 'scale' is usually the best default.

9.2 f_classif Feature Selection

Same ANOVA F-test as used in Random Forest. We select the top 10 features by F-score. StandardScaler is then applied AFTER selection — this is important because SVC is sensitive to feature scale and StandardScaler normalization is required for the RBF kernel to work well.

9.3 One-Hot Encoding

Unlike Random Forest (which uses LabelEncoder), SVC uses `pd.get_dummies()` to one-hot encode all categorical columns. This avoids introducing artificial ordinal relationships between categories (e.g., encoding Country as 0/1/2 implies an ordering that doesn't exist).

Hyperparameter Grid

```
param_grid = {
    "C": [0.1, 1, 10],          # Regularization strength
    "gamma": ["scale", "auto"]  # Kernel coefficient
}
```

Output Visualizations

- Bar chart: Accuracy before vs after f_classif reduction
- Confusion matrix: Heatmap for the after model
- User prediction: Custom employee profile

10. PCA + Logistic Regression — Feature Extraction

Cell 13–14 — PCA with Logistic Regression (Feature Extraction)

10.1 What is PCA?

Principal Component Analysis (PCA) is a Feature Extraction technique — unlike feature selection which picks existing features, PCA creates entirely new features (principal components) that are linear combinations of the original features. The components are ordered by how much variance they explain. The first component explains the most variance, the second explains the most remaining variance, and so on.

Difference from Feature Selection:

Feature Selection (Chi-Square, `f_classif`, Variance Threshold) keeps a subset of original features.

Feature Extraction (PCA) creates new transformed features from combinations of original features.

Both reduce dimensionality but in fundamentally different ways.

10.2 Why StandardScaler must come before PCA

PCA finds directions of maximum variance. If features are on different scales (e.g., Age in range 20-60 vs Internet_Speed in range 1-300), the features with larger scales will dominate PCA even if they are not more informative. StandardScaler brings all features to zero mean and unit variance before PCA.

10.3 Scree Plot — Choosing n_components (Fixed Version)

In the original notebook, `n_components=5` was hardcoded with no justification. The fixed version first fits PCA with all components, plots the explained variance ratio (scree plot) and cumulative variance curve, then automatically selects the number of components needed to retain 90% of the total variance.

```
pca_full = PCA()
pca_full.fit(X_scaled)

cumvar = np.cumsum(pca_full.explained_variance_ratio_)

# Plot scree plot + cumulative variance
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.bar(range(1, len(pca_full.explained_variance_ratio_)+1),
pca_full.explained_variance_ratio_)
plt.title("Scree Plot")

plt.subplot(1, 2, 2)
plt.plot(range(1, len(cumvar)+1), cumvar, marker='o')
plt.axhline(y=0.90, color='red', linestyle='--', label='90% variance')
plt.title("Cumulative Variance")
plt.show()

N_COMPONENTS = np.argmax(cumvar >= 0.90) + 1 # Auto-select
```

10.4 Baseline Model (Before PCA) — Fixed Version

The original notebook had no baseline model — it only ran the PCA model with no comparison. The fixed version adds a full Logistic Regression trained on the standardized but non-PCA data, so a valid before/after accuracy comparison is possible.

10.5 GridSearchCV on PCA Model — Fixed Version

The original model used default LogisticRegression parameters. The fixed version adds GridSearchCV to tune C and solver, consistent with all other models in the project.

Output Visualizations

- Scree plot: Individual explained variance per component
- Cumulative variance plot: Shows exactly where 80% and 90% thresholds are met
- Bar chart: Accuracy before PCA (full features) vs after PCA
- Confusion matrix: Heatmap for the PCA model

11. Dimension Reduction Techniques Summary

Technique				
Correlation Matrix	Feature Selection	Pre-analysis	Pearson r	Visual, interpretable
Variance Threshold	Feature Selection	Pre-processing	Feature variance	Removes constant features
Chi-Square (χ^2)	Feature Selection	LR + DT	χ^2 statistic	Good for categorical features
f_classif	Feature Selection	RF + SVC	F-statistic	Good for continuous features
PCA	Feature Extraction	LR (PCA model)	Explained variance	Reduces to orthogonal components

12. Hyperparameter Tuning — GridSearchCV

All models in this project use GridSearchCV for hyperparameter tuning. GridSearchCV performs an exhaustive search over a specified parameter grid, evaluating each combination using k-fold cross-validation (k=5 in this project). The combination that produces the highest mean accuracy across folds is selected as the best model.

Why cross-validation and not just train/test split?

If we tuned on the test set directly, we would overfit to the test set — the selected hyperparameters would be the best for THAT specific test split, not for new data. Cross-validation uses the training set only, splitting it 5 ways and rotating which fold is used as validation. This gives a more reliable estimate of generalization performance.

Model	Parameters Tuned	Values Searched
Logistic Regression	C, solver	C: [0.01, 0.1, 1, 10] solver: [liblinear, lbfgs]
Decision Tree	k, max_depth, min_samples_split	k: [5,10,15] depth: [5,10,15] split: [2,5,10]
Random Forest	n_estimators, max_depth	n_estimators: [50,100] max_depth: [5,10,None]
SVC	C, gamma	C: [0.1,1,10] gamma: [scale, auto]
PCA + LR	C, solver	C: [0.1,1,10] solver: [liblinear, lbfgs]

13. Original vs Fixed Notebook — What Changed

Issue	Status	Fix Applied
Correlation matrix missing	FIXED	Added full heatmap + top-10 target correlation table
VarianceThreshold imported but never used	FIXED	Now applied with threshold=0.01 and variance bar chart
PCA has no before-model baseline	FIXED	Added GridSearchCV Logistic Regression on full scaled features
PCA n_components=5 hardcoded	FIXED	Scree plot now auto-selects N for 90% explained variance
PCA model has no GridSearchCV	FIXED	Added GridSearchCV with C and solver parameters
SMOTE only in RF after, not before	FIXED	SMOTE applied consistently in both before and after splits
No boxplot for outlier visualization	FIXED	Before and after boxplots added in preprocessing section
Chi-Square on scaled data (invalid)	FIXED	MinMaxScaler applied before chi2, StandardScaler after

14. Project Links

Code Link

[Work Productivity / Burnout Risk](#)

Dataset Link

[Work Productivity.csv](#)

15. Conclusion

This project demonstrates a complete machine learning classification pipeline applied to a real-world problem: predicting employee burnout risk. The following was achieved:

Data Quality: The dataset was cleaned by removing nulls, duplicates, and outliers using the IQR method. Ordinal features were properly encoded using ordered numeric mapping.

Dimension Reduction: Five techniques were applied: Correlation Matrix (heatmap), Variance Threshold (filter), Chi-Square (statistical test for categorical features), `f_classif` (ANOVA F-test for numeric features), and PCA (feature extraction). Each technique was visualized and explained.

Model Comparison: Four classification models were trained and evaluated: Logistic Regression, Decision Tree, Random Forest, and SVC. Each model was run twice — once with all features (before) and once with dimension reduction applied (after) — to demonstrate the effect of reducing features.

PCA: PCA was used as a feature extraction technique paired with Logistic Regression. The scree plot was used to justify the number of components (those explaining $\geq 90\%$ cumulative variance), and a before/after comparison was properly added.

Hyperparameter Tuning: All five models used GridSearchCV with 5-fold cross-validation to find the optimal parameters, ensuring models were not evaluated on the same data used for tuning.

Results: In all models, accuracy after dimension reduction was equal to or higher than before, demonstrating that reducing features to the most informative ones does not hurt performance and can improve generalization by removing noise.

End of Report